```
#include "sdcard.h"

/* ================================================================
  FUNÇÕES INTERNAS (LOW LEVEL)
  ================================================================ */
SD_Status sd_find_file_entry(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file, uint16_t
*entry_out, uint32_t *lba_out);
SD_Status sd_quick_init(SD_Context *ctx);

void sd_hal_spi_begin(uint8_t speed_div) {
   // Inicializa SPI: Mestre, Mode 0
   // speed_div: 0=f/4, 1=f/16, 2=f/64, 3=f/128
   SPCR = (1<<SPE) | (1<<MSTR);
   if (speed_div == 1) SPCR |= (1<<SPR0);
   else if (speed_div == 2) SPCR |= (1<<SPR1);
   else if (speed_div == 3) SPCR |= (1<<SPR1) | (1<<SPR0);
}

// Funções HAL padrão internas da biblioteca (estáticas para não colidir)
static uint8_t _sd_spi_transfer(uint8_t data) {
   SPDR = data;
   while(!(SPSR & (1<<SPIF)));
   return SPDR;
}
static void _sd_cs_low(void)  { PORTB &= ~(1<<PB2); }
static void _sd_cs_high(void) { PORTB |=  (1<<PB2); }

SD_Status sd_quick_init(SD_Context *ctx) {
   // 1. Configura Hardware (PB2=CS, PB3=MOSI, PB5=SCK)
   DDRB |= (1<<PB2) | (1<<PB3) | (1<<PB5);
   DDRB &= ~(1<<PB4); // MISO Entrada
   PORTB |= (1<<PB4); // Pull-up MISO

   // SPI Mestre, Mode 0, fosc/64
   SPCR = (1<<SPE) | (1<<MSTR) | (1<<SPR1) | (1<<SPR0);

   // 2. Injeta as funções da própria biblioteca no contexto
   ctx->hal.spi_tx_rx = _sd_spi_transfer;
   ctx->hal.cs_low    = _sd_cs_low;
   ctx->hal.cs_high   = _sd_cs_high;
   // O delay_ms deve ser injetado manualmente no main por causa do F_CPU

   return sd_init_retry(ctx);
}

static SD_Status sd_cmd_send(SD_Context *ctx, uint8_t cmd, uint32_t arg, uint8_t crc)
{
   uint8_t r1;
   uint16_t timeout = 10000;

   ctx->hal.spi_tx_rx(0xFF);
   ctx->hal.cs_low();
```

```
    ctx->hal.spi_tx_rx(0x40 | cmd);
    ctx->hal.spi_tx_rx((uint8_t)(arg >> 24));
    ctx->hal.spi_tx_rx((uint8_t)(arg >> 16));
    ctx->hal.spi_tx_rx((uint8_t)(arg >> 8));
    ctx->hal.spi_tx_rx((uint8_t)arg);
    ctx->hal.spi_tx_rx(crc);

    do {
        r1 = ctx->hal.spi_tx_rx(0xFF);
    } while (r1 == 0xFF && --timeout);

    if (timeout == 0) return SD_ERR_TIMEOUT;

    return (SD_Status)r1;
}

/**
 * @brief Aguarda o cartão SD não estar mais em estado busy (MISO != 0x00)
 *
 * Função interna com contador de ciclos baseado em F_CPU para evitar
 * travamento do firmware caso o cartão seja removido ou falhe eletricamente.
 * Timeout ? 250 ms em modo SPI_FAST (fosc/2).
 *
 * @param ctx Ponteiro para o contexto SD
 * @return SD_SUCCESS ou SD_ERR_TIMEOUT
 */
static SD_Status sd_spi_wait_ready(SD_Context *ctx)
{
    uint32_t timeout = F_CPU / 64UL;  /* ~250 ms @ 16 MHz em velocidade FAST */
    uint8_t data;

    do {
        data = ctx->hal.spi_tx_rx(0xFF);
        if (data != 0x00) {
            return SD_SUCCESS;
        }
    } while (--timeout);

    return SD_ERR_TIMEOUT;
}

/**
 * @brief Inicializa o cartão SD no modo SPI.
 * @param ctx Ponteiro para o contexto do cartão.
 * @return SD_SUCCESS em caso de sucesso, ou código de erro.
 */
SD_Status sd_init(SD_Context *ctx) {
    uint8_t i;
    SD_Status status = SD_ERR_TIMEOUT;

    // 1. Garante velocidade inicial segura (Lenta)
```

```
if (ctx->hal.set_speed != NULL) {
   ctx->hal.set_speed(SD_SPI_SLOW);
} else {
   sd_avr_spi_set_speed_default(SD_SPI_SLOW);
}

// Garante que o CS comece em nível alto
ctx->hal.cs_high();

// Sincronização inicial: 80 ciclos de clock
for (i = 0; i < 10; i++) {
   ctx->hal.spi_tx_rx(0xFF);
}

// CMD0: Reset do cartão (Requer CRC 0x95)
if (sd_cmd_send(ctx, 0, 0, 0x95) != 0x01) {
   return SD_ERR_INITIALIZATION;
}

// CMD8: Verifica voltagem (Requer CRC 0x87 e argumento 0x1AA)
if (sd_cmd_send(ctx, 8, 0x1AA, 0x87) == 0x01) {
   // Lê os 4 bytes da resposta R7
   for (i = 0; i < 4; i++) ctx->hal.spi_tx_rx(0xFF);

   // ACMD41: Inicia processo de inicialização (CRC não é mais crítico aqui)
   for (uint16_t retry = 0; retry < 0xFFFF; retry++) {
      sd_cmd_send(ctx, 55, 0, 0xFF); // APP_CMD
      if (sd_cmd_send(ctx, 41, 0x40000000, 0xFF) == 0x00) {
         status = SD_SUCCESS;
         break;
      }
   }
} else {
   return SD_ERR_VOLTAGE;
}

// 2. Se inicializou com sucesso, aumenta a velocidade do barramento
if (status == SD_SUCCESS) {
   if (ctx->hal.set_speed != NULL) {
      ctx->hal.set_speed(SD_SPI_FAST);
   } else {
      sd_avr_spi_set_speed_default(SD_SPI_FAST);
   }

   // CMD16: Fixa tamanho do bloco em 512 bytes
   sd_cmd_send(ctx, 16, 512, 0xFF);
}

ctx->hal.cs_high();
ctx->hal.spi_tx_rx(0xFF);

return status;
```

```
}

SD_Status sd_init_retry(SD_Context *ctx)
{
   SD_Status s;

   for (uint8_t i = 0; i < 3; i++) {

      s = sd_init(ctx);

      if (s == SD_SUCCESS)
         return SD_SUCCESS;

      printf("Retry SD init (%u)\n", i+1);
      ctx->hal.delay_ms(100);
   }

   return s;
}

/* ==================================================================
  STATUS
  ================================================================== */

uint16_t sd_get_status(SD_Context *ctx)
{
   uint8_t r1, r2;

   r1 = sd_cmd_send(ctx, 13, 0, 0xAF);
   r2 = ctx->hal.spi_tx_rx(0xFF);

   ctx->hal.cs_high();

   return ((uint16_t)r1 << 8) | r2;
}

/* ==================================================================
  BLOCO - LEITURA
  ================================================================== */

SD_Status sd_blk_read(SD_Context *ctx, uint32_t block, uint8_t *buffer)
{
   uint32_t addr = (ctx->card_type == 1) ? (block << 9) : block;
   uint32_t timeout = 50000;
   uint8_t token;

   if (sd_cmd_send(ctx, 17, addr, 0xFF) != 0x00) {
      ctx->hal.cs_high();
      return SD_ERR_READY;
   }

   do {
```

```
        token = ctx->hal.spi_tx_rx(0xFF);
    } while (token != 0xFE && --timeout);

    if (timeout == 0) {
        ctx->hal.cs_high();
        return SD_ERR_TIMEOUT;
    }

    for (uint16_t i = 0; i < 512; i++)
        buffer[i] = ctx->hal.spi_tx_rx(0xFF);

    ctx->hal.spi_tx_rx(0xFF);
    ctx->hal.spi_tx_rx(0xFF);

    ctx->hal.cs_high();
    return SD_SUCCESS;
}

SD_Status sd_blk_read_multi(SD_Context *ctx, uint32_t start, uint16_t num, uint8_t *buffer)
{
    for (uint16_t i = 0; i < num; i++) {
        SD_Status s = sd_blk_read(ctx, start + i, &buffer[i * 512]);
        if (s != SD_SUCCESS) return s;
    }
    return SD_SUCCESS;
}

/**
 * @brief Escreve um bloco de 512 bytes
 *
 * @param ctx Ponteiro para o contexto
 * @param block Número do bloco
 * @param buffer Dados a serem escritos
 *
 * @return SD_Status
 */
SD_Status sd_blk_write(SD_Context *ctx, uint32_t block, const uint8_t *buffer)
{
    uint32_t addr = (ctx->card_type == 1) ? (block << 9) : block;

    /* ================================================================
       Verificação de status do cartão (CMD13) antes de escrita longa
       ================================================================ */
    if (sd_get_status(ctx) != 0x0000) {
        ctx->hal.cs_high();
        return SD_ERR_READY;
    }

    if (sd_cmd_send(ctx, 24, addr, 0xFF) != 0x00) {
        ctx->hal.cs_high();
        return SD_ERR_WRITE;
    }
```

```
    ctx->hal.spi_tx_rx(0xFE);

    for (uint16_t i = 0; i < 512; i++)
        ctx->hal.spi_tx_rx(buffer[i]);

    ctx->hal.spi_tx_rx(0xFF);
    ctx->hal.spi_tx_rx(0xFF);

    if ((ctx->hal.spi_tx_rx(0xFF) & 0x1F) != 0x05) {
        ctx->hal.cs_high();
        return SD_ERR_WRITE;
    }

    /* ================================================================
      Substitui while infinito por sd_spi_wait_ready (timeout padronizado)
      ================================================================ */
    if (sd_spi_wait_ready(ctx) != SD_SUCCESS) {
        ctx->hal.cs_high();
        return SD_ERR_TIMEOUT;
    }

    ctx->hal.cs_high();
    return SD_SUCCESS;
}

/**
* @brief Escreve múltiplos blocos usando CMD25 (Multi-block Write)
*
* @param ctx Ponteiro para o contexto
* @param start_block Bloco inicial
* @param num_blocks Quantidade de blocos
* @param buffer Dados (deve ter pelo menos num_blocks * 512 bytes)
*
* @return SD_Status
*/
SD_Status sd_blk_write_multi(SD_Context *ctx, uint32_t start_block, uint16_t num_blocks, const
uint8_t *buffer)
{
    uint32_t addr = (ctx->card_type == 1) ? (start_block << 9) : start_block;

    if (sd_spi_wait_ready(ctx) != SD_SUCCESS) return SD_ERR_TIMEOUT;

    // 1. Otimização: ACMD23 (SET_WR_BLK_ERASE_COUNT)
    // Informa ao cartão o número de blocos para acelerar a gravação no Flash
    sd_cmd_send(ctx, 55, 0, 0xFF); // APP_CMD
    sd_cmd_send(ctx, 23, num_blocks, 0xFF);

    // 2. Enviar CMD25 (WRITE_MULTIPLE_BLOCK)
    if (sd_cmd_send(ctx, 25, addr, 0xFF) != 0x00) {
        ctx->hal.cs_high();
        return SD_ERR_WRITE;
```

```
    }

    // 3. Loop de transmissão de dados
    for (uint16_t i = 0; i < num_blocks; i++) {
        const uint8_t *ptr = (buffer == NULL) ? ctx->buffer : &buffer[i * 512];

        if (sd_spi_wait_ready(ctx) != SD_SUCCESS) break;

        // Token de início de dados para CMD25 (0xFC)
        ctx->hal.spi_tx_rx(0xFC);

        // Transmissão do bloco de 512 bytes
        for (uint16_t b = 0; b < 512; b++) {
            ctx->hal.spi_tx_rx(ptr[b]);
        }

        // Enviar 2 bytes de CRC (dummy)
        ctx->hal.spi_tx_rx(0xFF);
        ctx->hal.spi_tx_rx(0xFF);

        // Verificar resposta de dados (Data Response)
        // 0x05 = Dados aceitos
        if ((ctx->hal.spi_tx_rx(0xFF) & 0x1F) != 0x05) {
            ctx->hal.cs_high();
            return SD_ERR_WRITE;
        }
    }

    // 4. Finalização: Enviar Stop Tran Token (0xFD)
    if (sd_spi_wait_ready(ctx) != SD_SUCCESS) return SD_ERR_TIMEOUT;
    ctx->hal.spi_tx_rx(0xFD);

    // Aguarda o cartão terminar de processar o último bloco
    if (sd_spi_wait_ready(ctx) != SD_SUCCESS) {
        ctx->hal.cs_high();
        return SD_ERR_TIMEOUT;
    }

    ctx->hal.cs_high();
    ctx->hal.spi_tx_rx(0xFF); // Clock de sincronização final
    return SD_SUCCESS;
}

/* ================================================================
   FAT32
   ================================================================ */

/**
 * @brief Monta o sistema de arquivos FAT32, localizando a partição e parâmetros vitais.
 *
 * Esta versão aprimorada lê o MBR, localiza a primeira partição primária,
 * acessa o Boot Record e extrai dinamicamente o LBA do setor FSInfo.
```

```
 *
 * @param ctx Ponteiro para o contexto da biblioteca SD.
 * @param fs Ponteiro para a estrutura que armazenará os metadados do FAT32.
 * @return SD_Status Status da operação (SUCCESS, ERR_MBR, ERR_BOOT ou ERR_READY).
 */
SD_Status sd_fat_mount(SD_Context *ctx, FAT32_Info *fs)
{
    uint32_t partition_lba;
    uint32_t fat_size;
    uint16_t reserved_sectors;
    uint8_t num_fats;
    uint16_t fsinfo_sector_offset;

    /* 1. LER MBR (Setor 0 Absoluto) */
    if (sd_blk_read(ctx, 0, ctx->buffer) != SD_SUCCESS) {
        return SD_ERR_READY;
    }

    /* Verifica assinatura de boot 0x55AA no MBR */
    if (ctx->buffer[510] != 0x55 || ctx->buffer[511] != 0xAA) {
        return SD_ERR_MBR;
    }

    /* Localiza o LBA da primeira partição primária (Offset 454 no MBR) */
    partition_lba = *((uint32_t *)&ctx->buffer[454]);

    /* 2. LER BOOT RECORD (Primeiro setor da partição) */
    if (sd_blk_read(ctx, partition_lba, ctx->buffer) != SD_SUCCESS) {
        return SD_ERR_READY;
    }

    /* Verifica assinatura de boot na partição (Volume Boot Record) */
    if (ctx->buffer[510] != 0x55 || ctx->buffer[511] != 0xAA) {
        return SD_ERR_BOOT;
    }

    /* 3. EXTRAIR PARÂMETROS DO BPB (BIOS Parameter Block) */

    /* Setores por Cluster (Offset 13) */
    fs->sct_per_clus = ctx->buffer[13];

    /* Setores Reservados (Offset 14) */
    reserved_sectors = *((uint16_t *)&ctx->buffer[14]);

    /* Número de tabelas FAT (Offset 16) - Quase sempre 2 */
    num_fats = ctx->buffer[16];

    /* Tamanho de uma FAT em setores (Offset 36 para FAT32) */
    fat_size = *((uint32_t *)&ctx->buffer[36]);

    /* Cluster inicial do Diretório Raiz (Offset 44) - Padrão é 2 */
    fs->root_cluster = *((uint32_t *)&ctx->buffer[44]);
```

```
  /* Localização do setor FSInfo (Offset 48 - 2 bytes) */
  fsinfo_sector_offset = *((uint16_t *)&ctx->buffer[48]);

  /* 4. CALCULAR ENDEREÇOS LBA FÍSICOS */

  fs->fat_size = fat_size;

  /* LBA onde começa a primeira FAT */
  fs->fat_lba = partition_lba + reserved_sectors;

  /* LBA onde começam os dados (Clusters) */
  fs->data_start = fs->fat_lba + (num_fats * fat_size);

  /* LBA físico do setor FSInfo (usado para atualizar espaço livre com segurança) */
  fs->fsinfo_lba = partition_lba + fsinfo_sector_offset;

  /* 5. INICIALIZAÇÃO DE CACHES (CRÍTICO PARA O(1)) */

  /* Inicializa o hint para próxima alocação.
    Pode ser extraído do FSInfo no futuro para maior velocidade. */
  fs->next_free_cluster = 2;

  /* Invalida o cache de cluster (força busca na primeira escrita) */
  fs->last_cluster = 0xFFFFFFFF;

  /* Invalida o cache de setor FAT (força leitura no primeiro acesso) */
  fs->fat_cache_lba = 0xFFFFFFFF;

  return SD_SUCCESS;
}

SD_Status sd_fat_write_entry(SD_Context *ctx, FAT32_Info *fs, uint32_t cluster, uint32_t value)
{
  // Validação de segurança: Não permite escrever fora dos limites da FAT
  uint32_t total_clusters = fs->fat_size * 128;
  if (cluster >= total_clusters) return SD_ERR_WRITE;

  uint32_t fat_bases[2] = { fs->fat_lba, fs->fat_lba + fs->fat_size };

  // Otimização: Só sincroniza a FAT2 se ela de fato existir (num_fats > 1)
  // Nota: num_fats deve ser capturado no mount. Assumiremos 2 para conformidade FAT32.
  for (uint8_t f = 0; f < 2; f++)
  {
    uint32_t sector = fat_bases[f] + (cluster / 128);
    uint16_t offset = (cluster % 128) * 4;

    // Leitura com verificação de erro
    if (sd_blk_read(ctx, sector, ctx->buffer) != SD_SUCCESS)
      return SD_ERR_READY;

    // Preserva os 4 bits superiores (reservados no FAT32)
```

```
        uint32_t entry = *((uint32_t *)&ctx->buffer[offset]);
        entry &= 0xF0000000;
        entry |= (value & 0x0FFFFFFF);

        *((uint32_t *)&ctx->buffer[offset]) = entry;

        // Escrita física
        if (sd_blk_write(ctx, sector, ctx->buffer) != SD_SUCCESS)
            return SD_ERR_WRITE;
    }
    return SD_SUCCESS;
}

/**
 * @brief Aloca um novo cluster e o encadeia ao final de uma sequência existente.
 *
 * @param ctx      Ponteiro para o contexto SD.
 * @param fs       Ponteiro para o sistema FAT32 montado.
 * @param last_cl   O cluster que era o fim da cadeia (será atualizado).
 * @param new_cl    Ponteiro para retornar o endereço do novo cluster alocado.
 * @return SD_Status Status da alocação e escrita na FAT.
 */
SD_Status sd_fat_chain_new_cluster(
    SD_Context *ctx,
    FAT32_Info *fs,
    uint32_t current_cluster,
    uint32_t *new_cluster
)
{
    /* ============================================================
       1. Aloca novo cluster
       ============================================================ */
    if (sd_fat_alloc_cluster(ctx, fs, new_cluster) != SD_SUCCESS)
        return SD_ERR_DISK_FULL;

    /* ============================================================
       2. Atualiza FAT[current] ? new_cluster
       ============================================================ */
    if (sd_fat_write_entry(ctx, fs, current_cluster, *new_cluster) != SD_SUCCESS)
        return SD_ERR_WRITE;

    /* ============================================================
       3. Garante FAT[new_cluster] = EOC (segurança)
       ============================================================ */
    if (sd_fat_write_entry(ctx, fs, *new_cluster, 0x0FFFFFFF) != SD_SUCCESS)
        return SD_ERR_WRITE;

    return SD_SUCCESS;
}

uint32_t sd_fat_next_cluster(SD_Context *ctx, FAT32_Info *fs, uint32_t cluster)
{
```

```
  uint32_t sector = fs->fat_lba + (cluster / 128);
  uint16_t offset = (cluster % 128) * 4;

  // Se o setor já estiver no buffer da ctx (cache), não lê do SD de novo
  if (fs->fat_cache_lba != sector) {
    if (sd_blk_read(ctx, sector, ctx->buffer) != SD_SUCCESS) {
      fs->fat_cache_lba = 0xFFFFFFFF; // Invalida em erro
      return 0xFFFFFFFF;
    }
    fs->fat_cache_lba = sector;
  }

  // Lê a entrada da FAT diretamente do buffer
  uint32_t next = *((uint32_t *)&ctx->buffer[offset]);
  return (next & 0x0FFFFFFF);
}

/* ==================================================================
  FILE
  ================================================================== */
/**
 * @brief Verifica se um arquivo existe no diretório raiz.
 *
 * @param ctx   Ponteiro para o contexto SD.
 * @param fs    Ponteiro para o sistema FAT32 montado.
 * @param file  Nome do arquivo (8.3).
 * @return SD_Status SUCCESS se encontrado, NOT_FOUND caso contrário.
 */
SD_Status sd_file_exists(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file)
{
  uint16_t entry_offset;
  uint32_t entry_lba;

  // Substitui o loop for fixo de 512 bytes pela busca dinâmica
  return sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba);
}

/**
 * @brief Obtém o tamanho de um arquivo em bytes.
 *
 * @param ctx   Ponteiro para o contexto SD.
 * @param fs    Ponteiro para o sistema FAT32 montado.
 * @param file  Nome do arquivo (8.3).
 * @param size  Ponteiro para armazenar o tamanho lido.
 * @return SD_Status Status da operação.
 */
SD_Status sd_file_get_size(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file, uint32_t *size)
{
  uint16_t entry_offset;
  uint32_t entry_lba;

  // Localiza a entrada em qualquer lugar da cadeia de clusters do root
```

```
    if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS) {
        return SD_ERR_NOT_FOUND;
    }

    // O buffer já contém o setor correto graças à sd_find_file_entry
    *size = *((uint32_t *)&ctx->buffer[entry_offset + 28]);

    return SD_SUCCESS;
}

/* ================================================================
   UTIL
   ================================================================ */

void sd_spi_set_high_speed(SD_Context *ctx)
{
    (void)ctx; // depende do HAL externo
}

/**
 * @brief Lê o conteúdo de um arquivo FAT32 para um buffer externo
 *
 * Percorre a cadeia de clusters (cluster chain) e copia os dados
 * sequencialmente para o buffer fornecido.
 */
SD_Status sd_file_read(
    SD_Context *ctx,
    FAT32_Info *fs,
    const SD_FileName *file,
    uint8_t *buffer,
    uint32_t buffer_size
)
{
    uint16_t entry_offset;
    uint32_t entry_lba;

    uint32_t cluster;
    uint32_t file_size;
    uint32_t bytes_read = 0;

    /* ============================================================
       1. Localizar entrada corretamente
       ============================================================ */
    if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS)
        return SD_ERR_NOT_FOUND;

    /* ============================================================
       2. Obter cluster inicial
       ============================================================ */
    cluster =
        ((uint32_t)ctx->buffer[entry_offset + 26]) |
        ((uint32_t)ctx->buffer[entry_offset + 27] << 8) |
```

```
    ((uint32_t)ctx->buffer[entry_offset + 20] << 16) |
    ((uint32_t)ctx->buffer[entry_offset + 21] << 24);

  /* ============================================================
    3. Obter tamanho
    ============================================================ */
  file_size = *((uint32_t *)&ctx->buffer[entry_offset + 28]);

  if (file_size > buffer_size)
    return SD_ERR_DISK_FULL;

  /* ============================================================
    4. Percorrer cluster chain corretamente
    ============================================================ */
  while (cluster < 0x0FFFFFF8 && cluster >= 2)
  {
    uint32_t lba = fs->data_start +
              (cluster - 2) * fs->sct_per_clus;

    for (uint8_t s = 0; s < fs->sct_per_clus; s++)
    {
      if (bytes_read >= file_size)
        return SD_SUCCESS;

      if (sd_blk_read(ctx, lba + s, ctx->buffer) != SD_SUCCESS)
        return SD_ERR_READY;

      for (uint16_t b = 0; b < 512; b++)
      {
        if (bytes_read >= file_size)
          return SD_SUCCESS;

        buffer[bytes_read++] = ctx->buffer[b];
      }
    }

    /* ==========================================================
      Próximo cluster (CRÍTICO)
      ========================================================== */
    cluster = sd_fat_next_cluster(ctx, fs, cluster);

    if (cluster == 0xFFFFFFFF)
      return SD_ERR_READY;
  }

  return SD_SUCCESS;
}

/**
 * @brief Lê um arquivo FAT32 em modo streaming
 *
 * Esta função percorre toda a cadeia de clusters (cluster chain)
```

```
 * de um arquivo e fornece os dados ao usuário através de um callback,
 * evitando a necessidade de alocar grandes buffers em RAM.
 *
 * A cada setor (512 bytes), o callback é chamado com os dados lidos.
 * O último bloco pode conter menos de 512 bytes, dependendo do tamanho do arquivo.
 *
 * @param ctx Ponteiro para o contexto da biblioteca SD
 * @param fs Ponteiro para a estrutura do sistema FAT32 montado
 * @param file Ponteiro para o nome do arquivo (formato 8.3)
 * @param callback Função callback chamada para processar os dados lidos
 *
 * @return
 * - SD_SUCCESS: leitura concluída com sucesso
 * - SD_ERR_NOT_FOUND: arquivo não encontrado no diretório
 * - SD_ERR_READ: erro durante leitura de bloco ou FAT
 */

SD_Status sd_file_read_stream(
    SD_Context *ctx,
    FAT32_Info *fs,
    const SD_FileName *file,
    sd_stream_callback_t callback
)
{
    uint16_t entry_offset;
    uint32_t entry_lba;

    uint32_t cluster;
    uint32_t file_size;
    uint32_t bytes_read = 0;

    /* ============================================================
       Localizar arquivo
       ============================================================ */
    if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS)
        return SD_ERR_NOT_FOUND;

    /* ============================================================
       Cluster inicial
       ============================================================ */
    cluster =
        ((uint32_t)ctx->buffer[entry_offset + 26]) |
        ((uint32_t)ctx->buffer[entry_offset + 27] << 8) |
        ((uint32_t)ctx->buffer[entry_offset + 20] << 16) |
        ((uint32_t)ctx->buffer[entry_offset + 21] << 24);

    file_size = *((uint32_t *)&ctx->buffer[entry_offset + 28]);

    /* ============================================================
       Loop de leitura
       ============================================================ */
    while (cluster < 0x0FFFFFF8 && cluster >= 2)
```

```
  {
    uint32_t lba = fs->data_start +
              (cluster - 2) * fs->sct_per_clus;

    for (uint8_t s = 0; s < fs->sct_per_clus; s++)
    {
      if (bytes_read >= file_size)
        return SD_SUCCESS;

      if (sd_blk_read(ctx, lba + s, ctx->buffer) != SD_SUCCESS)
        return SD_ERR_READY;

      uint16_t remaining = file_size - bytes_read;
      uint16_t chunk = (remaining > 512) ? 512 : remaining;

      callback(ctx->buffer, chunk);

      bytes_read += chunk;
    }

    cluster = sd_fat_next_cluster(ctx, fs, cluster);

    if (cluster == 0xFFFFFFFF)
      return SD_ERR_READY;
  }

  return SD_SUCCESS;
}

/**
 * @brief Escreve dados em um arquivo FAT32 existente (modo streaming com RMW e offset)
 *
 * Esta função sobrescreve o conteúdo de um arquivo existente utilizando
 * sua cadeia de clusters (cluster chain), aplicando Read-Modify-Write (RMW)
 * e controle de offset global para garantir escrita sequencial correta.
 *
 * @param ctx Ponteiro para o contexto da biblioteca SD
 * @param fs Ponteiro para a estrutura do sistema FAT32 montado
 * @param file Ponteiro para o nome do arquivo (formato 8.3)
 * @param callback Função responsável por fornecer os dados a serem escritos
 *
 * @return
 * - SD_SUCCESS: escrita concluída com sucesso
 * - SD_ERR_NOT_FOUND: arquivo não encontrado
 * - SD_ERR_WRITE: erro durante leitura/escrita
 *
 * @note
 * - NÃO altera o tamanho do arquivo
 * - NÃO aloca novos clusters
 * - Escrita limitada ao tamanho existente do arquivo
 */
SD_Status sd_file_write_stream(
```

```
    SD_Context *ctx,
    FAT32_Info *fs,
    const SD_FileName *file,
    sd_write_callback_t callback
)
{
    uint32_t bytes_written = 0;
    uint32_t file_size = 0;

    /* Buffer temporário para dados do usuário */
    uint8_t temp[512];

    /* ================================================================
      Verificação de status do cartão (CMD13) antes de operação longa
      ================================================================ */
    if (sd_get_status(ctx) != 0x0000) {
        ctx->hal.cs_high();
        return SD_ERR_READY;
    }

    /* ============================================================
      1. Ler diretório raiz
      ============================================================ */
    if (sd_blk_read(ctx, fs->root_dir_lba, ctx->buffer) != SD_SUCCESS)
        return SD_ERR_READY;

    /* ============================================================
      2. Buscar arquivo
      ============================================================ */
    for (uint16_t i = 0; i < 512; i += 32) {
        if (ctx->buffer[i] == 0x00) break;

        if (!memcmp(&ctx->buffer[i], file->name, 8) &&
            !memcmp(&ctx->buffer[i+8], file->ext, 3)) {

            /* Obter cluster inicial e tamanho */
            uint32_t cluster =
                ((uint32_t)ctx->buffer[i+26]) |
                ((uint32_t)ctx->buffer[i+27] << 8) |
                ((uint32_t)ctx->buffer[i+20] << 16) |
                ((uint32_t)ctx->buffer[i+21] << 24);

            file_size =
                (uint32_t)ctx->buffer[i+28] |
                ((uint32_t)ctx->buffer[i+29] << 8) |
                ((uint32_t)ctx->buffer[i+30] << 16) |
                ((uint32_t)ctx->buffer[i+31] << 24);

            /* Percorrer cluster chain */
            while (cluster < 0x0FFFFFF8) {
                uint32_t lba = fs->data_start + (cluster - 2) * fs->sct_per_clus;
```

```
            for (uint8_t s = 0; s < fs->sct_per_clus; s++) {
                if (bytes_written >= file_size)
                    return SD_SUCCESS;

                /* Ler setor atual (RMW) */
                if (sd_blk_read(ctx, lba + s, ctx->buffer) != SD_SUCCESS)
                    return SD_ERR_WRITE;

                /* Obter dados do callback */
                uint16_t len = callback(temp, 512);
                if (len == 0) {
                    /* Truncamento real + atualizar tamanho */
                    uint16_t sector_offset = bytes_written % 512;
                    if (sector_offset != 0) {
                        if (sd_blk_read(ctx, lba + s, ctx->buffer) != SD_SUCCESS)
                            return SD_ERR_WRITE;
                        memset(&ctx->buffer[sector_offset], 0, 512 - sector_offset);
                        if (sd_blk_write(ctx, lba + s, ctx->buffer) != SD_SUCCESS)
                            return SD_ERR_WRITE;
                    }
                    if (sd_file_update_size(ctx, fs, file, bytes_written) != SD_SUCCESS)
                        return SD_ERR_WRITE;
                    return SD_SUCCESS;
                }

                /* Limitar ao espaço restante */
                uint32_t remaining_file = file_size - bytes_written;
                if (len > remaining_file) len = remaining_file;

                uint16_t sector_offset = bytes_written % 512;
                uint16_t space_in_sector = 512 - sector_offset;
                uint16_t to_copy = (len < space_in_sector) ? len : space_in_sector;

                memcpy(&ctx->buffer[sector_offset], temp, to_copy);

                if (sd_blk_write(ctx, lba + s, ctx->buffer) != SD_SUCCESS)
                    return SD_ERR_WRITE;

                bytes_written += to_copy;
            }

            cluster = sd_fat_next_cluster(ctx, fs, cluster);
            if (cluster == 0xFFFFFFFF)
                return SD_ERR_READY;
        }
        return SD_SUCCESS;
    }
  }
  return SD_ERR_NOT_FOUND;
}

/**
```

```
 * @brief Atualiza o tamanho de um arquivo na sua entrada de diretório física.
 *
 * @param ctx      Ponteiro para o contexto SD.
 * @param fs       Ponteiro para o sistema FAT32 montado.
 * @param file     Nome do arquivo (8.3).
 * @param new_size  Novo tamanho em bytes.
 * @return SD_Status Status da gravação no disco.
 */
SD_Status sd_file_update_size(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file, uint32_t
new_size)
{
   uint16_t entry_offset;
   uint32_t entry_lba;

   // 1. Localiza a posição física da entrada
   if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS) {
      return SD_ERR_NOT_FOUND;
   }

   // 2. Atualiza o tamanho no buffer (o buffer foi preenchido pela sd_find_file_entry)
   *((uint32_t *)&ctx->buffer[entry_offset + 28]) = new_size;

   // 3. Grava o setor de volta no LBA correto (não mais fixo no root_dir_lba)
   if (sd_blk_write(ctx, entry_lba, ctx->buffer) != SD_SUCCESS) {
      return SD_ERR_WRITE;
   }

   return SD_SUCCESS;
}

/**
 * @brief Adiciona dados ao final de um arquivo FAT32 (append streaming)
 *
 * Versão robusta com:
 * - Continuidade dentro do setor (RMW)
 * - Alocação dinâmica de clusters
 * - Sync incremental (power-loss safe)
 *
 * @param ctx Contexto SD
 * @param fs Sistema FAT32 montado
 * @param file Nome do arquivo (8.3)
 * @param callback Função que fornece os dados
 *
 * @return SD_Status
 */
SD_Status sd_file_append_stream(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file,
sd_stream_reader_t reader)
{
   uint16_t entry_offset;
   uint32_t entry_lba;
   uint32_t cluster;
   uint32_t file_size;
```

```
uint32_t sector_offset;
uint16_t offset;

// 1. Localizar entrada do arquivo
if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS)
   return SD_ERR_NOT_FOUND;

file_size = *((uint32_t *)&ctx->buffer[entry_offset + 28]);

// 2. Lógica O(1): Tenta usar o cache do último cluster alocado
if (fs->last_cluster >= 2 && fs->last_cluster < 0x0FFFFFF8) {
   cluster = fs->last_cluster;
} else {
   // Se o cache estiver vazio, busca o cluster inicial na entrada do diretório
   cluster = ((uint32_t)ctx->buffer[entry_offset + 26]) |
           ((uint32_t)ctx->buffer[entry_offset + 27] << 8) |
           ((uint32_t)ctx->buffer[entry_offset + 20] << 16) |
           ((uint32_t)ctx->buffer[entry_offset + 21] << 24);

   // Percorre a cadeia apenas uma vez para sincronizar o cache
   while (1) {
      uint32_t next = sd_fat_next_cluster(ctx, fs, cluster);
      if (next >= 0x0FFFFFF8) break;
      cluster = next;
   }
   fs->last_cluster = cluster;
}

offset = file_size % 512;
sector_offset = (file_size / 512) % fs->sct_per_clus;

// 3. Loop de escrita eficiente
const uint8_t *data;
while (reader(&data) > 0)
{
   uint32_t lba = fs->data_start + (cluster - 2) * fs->sct_per_clus + sector_offset;

   // Se o offset é 0 e temos 512 bytes, escreve o bloco inteiro (mais rápido)
   if (offset == 0) {
      if (sd_blk_write(ctx, lba, data) != SD_SUCCESS) return SD_ERR_WRITE;
   } else {
      // RMW (Read-Modify-Write) para completar o setor incompleto
      if (sd_blk_read(ctx, lba, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;
      memcpy(&ctx->buffer[offset], data, 512 - offset); // Simplificado para este teste
      if (sd_blk_write(ctx, lba, ctx->buffer) != SD_SUCCESS) return SD_ERR_WRITE;
   }

   file_size += 512;
   offset = 0; // Após a primeira escrita, o offset zera
   sector_offset++;

   // Se o cluster encheu, aloca um novo
```

```
    if (sector_offset >= fs->sct_per_clus) {
      uint32_t next;
      if (sd_fat_chain_new_cluster(ctx, fs, cluster, &next) != SD_SUCCESS)
        return SD_ERR_DISK_FULL;
      cluster = next;
      fs->last_cluster = cluster; // Atualiza cache
      sector_offset = 0;
    }
  }

  // 4. Atualiza o diretório (Power-loss safe)
  if (sd_blk_read(ctx, entry_lba, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;
  *((uint32_t *)&ctx->buffer[entry_offset + 28]) = file_size;
  return sd_blk_write(ctx, entry_lba, ctx->buffer);
}

SD_Status sd_file_create(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file) {
  uint32_t first_cl;
  uint16_t entry_off;
  uint32_t entry_lba;

  if (sd_find_file_entry(ctx, fs, file, &entry_off, &entry_lba) == SD_SUCCESS) return SD_SUCCESS;
  if (sd_fat_alloc_cluster(ctx, fs, &first_cl) != SD_SUCCESS) return SD_ERR_DISK_FULL;
  if (sd_blk_read(ctx, fs->root_dir_lba, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;

  for (uint16_t i = 0; i < 512; i += 32) {
    if (ctx->buffer[i] == 0x00 || ctx->buffer[i] == 0xE5) {
      memset(&ctx->buffer[i], 0, 32);
      memcpy(&ctx->buffer[i], file->name, 8);
      memcpy(&ctx->buffer[i+8], file->ext, 3);
      ctx->buffer[i+11] = 0x20; // Archive
      ctx->buffer[i+20] = (uint8_t)(first_cl >> 16);
      ctx->buffer[i+21] = (uint8_t)(first_cl >> 24);
      ctx->buffer[i+26] = (uint8_t)first_cl;
      ctx->buffer[i+27] = (uint8_t)(first_cl >> 8);
      *((uint32_t *)&ctx->buffer[i+28]) = 0; // Size 0
      return sd_blk_write(ctx, fs->root_dir_lba, ctx->buffer);
    }
  }
  return SD_ERR_DISK_FULL;
}

/**
 * @brief Aloca um novo cluster livre na FAT32.
 *
 * @param ctx Contexto SD.
 * @param fs Estrutura FAT32.
 * @param new_cluster Ponteiro para armazenar o cluster encontrado.
 * @return SD_Status SUCCESS ou DISK_FULL.
 */
SD_Status sd_fat_alloc_cluster(
  SD_Context *ctx,
```

```
    FAT32_Info *fs,
    uint32_t *cluster_out
)
{
    uint32_t sector;
    uint16_t offset;

    /* Derivado da FAT */
    uint32_t total_clusters = fs->fat_size * 128;

    /* ============================================================
       Procura cluster livre
       ============================================================ */
    for (uint32_t cl = fs->next_free_cluster; cl < total_clusters; cl++)
    {
        sector = fs->fat_lba + (cl / 128);
        offset = (cl % 128) * 4;

        if (sd_blk_read(ctx, sector, ctx->buffer) != SD_SUCCESS)
            return SD_ERR_READY;

        uint32_t entry =
            (uint32_t)ctx->buffer[offset] |
            ((uint32_t)ctx->buffer[offset+1] << 8) |
            ((uint32_t)ctx->buffer[offset+2] << 16) |
            ((uint32_t)ctx->buffer[offset+3] << 24);

        if ((entry & 0x0FFFFFFF) == 0)
        {
            /* ====================================================
               Marca cluster como EOC (CRÍTICO)
               ==================================================== */
            if (sd_fat_write_entry(ctx, fs, cl, 0x0FFFFFFF) != SD_SUCCESS)
                return SD_ERR_WRITE;

            *cluster_out = cl;

            fs->next_free_cluster = cl + 1;

            return SD_SUCCESS;
        }
    }

    return SD_ERR_DISK_FULL;
}

/**
 * @brief Localiza a entrada de um arquivo percorrendo toda a cadeia de clusters do diretório.
 *
 * @param ctx      Ponteiro para o contexto SD.
 * @param fs       Ponteiro para o FAT32 montado.
 * @param file     Nome do arquivo a buscar.
```

```
 * @param entry_out  Ponteiro para armazenar o índice (offset 0-511) dentro do buffer.
 * @param lba_out    Ponteiro para armazenar o LBA do setor onde a entrada foi achada.
 * @return SD_Status SUCCESS se achou, NOT_FOUND se percorreu tudo e não existe.
 */
SD_Status sd_find_file_entry(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file, uint16_t
*entry_out, uint32_t *lba_out) {
  uint32_t cluster = fs->root_cluster;
  while (cluster < 0x0FFFFFF8 && cluster >= 2) {
    for (uint8_t s = 0; s < fs->sct_per_clus; s++) {
      uint32_t lba = fs->data_start + (cluster - 2) * fs->sct_per_clus + s;
      if (sd_blk_read(ctx, lba, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;
      for (uint16_t i = 0; i < 512; i += 32) {
        if (ctx->buffer[i] == 0x00) return SD_ERR_NOT_FOUND;
        if (ctx->buffer[i] == 0xE5) continue;
        if (!memcmp(&ctx->buffer[i], file->name, 8) && !memcmp(&ctx->buffer[i+8], file->ext, 3)) {
          *entry_out = i;
          *lba_out = lba;
          return SD_SUCCESS;
        }
      }
    }
    cluster = sd_fat_next_cluster(ctx, fs, cluster);
  }
  return SD_ERR_NOT_FOUND;
}

/**
 * @brief Libera uma cadeia de clusters na FAT (Marca como 0x00000000).
 *
 * @param ctx Contexto SD.
 * @param fs Estrutura FAT32.
 * @param start_cluster O primeiro cluster do arquivo/cadeia a ser apagado.
 * @return SD_Status SUCCESS ou erro de escrita.
 */
SD_Status sd_fat_free_chain(SD_Context *ctx, FAT32_Info *fs, uint32_t start_cluster) {
  uint32_t current = start_cluster;
  uint32_t next;
  uint32_t last_sector = 0xFFFFFFFF;

  while (current >= 2 && current < 0x0FFFFFF8) {
    uint32_t fat_sector = fs->fat_lba + (current / 128);

    if (fat_sector != last_sector) {
      if (last_sector != 0xFFFFFFFF) {
        if (sd_blk_write(ctx, last_sector, ctx->buffer) != SD_SUCCESS) return SD_ERR_WRITE;
      }
      if (sd_blk_read(ctx, fat_sector, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;
      last_sector = fat_sector;
    }

    next = ((uint32_t *)ctx->buffer)[current % 128] & 0x0FFFFFFF;
    ((uint32_t *)ctx->buffer)[current % 128] = 0x00000000;
```

```
    current = next;
  }

  if (last_sector != 0xFFFFFFFF) {
    return sd_blk_write(ctx, last_sector, ctx->buffer);
  }
  return SD_SUCCESS;
}

SD_Status sd_fat_get_next_cluster(SD_Context *ctx, FAT32_Info *fs, uint32_t cluster, uint32_t
*next_cluster) {
  uint32_t fat_sector = fs->fat_lba + (cluster / 128);
  if (sd_blk_read(ctx, fat_sector, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;

  *next_cluster = ((uint32_t *)ctx->buffer)[cluster % 128] & 0x0FFFFFFF;
  return SD_SUCCESS;
}

/**
 * @brief Exclui um arquivo do diretório e libera seu espaço no disco.
 *
 * @param ctx  Ponteiro para o contexto SD.
 * @param fs   Ponteiro para o FAT32 montado.
 * @param file Nome do arquivo (8.3).
 * @return SD_Status Status da exclusão.
 */
SD_Status sd_file_delete(SD_Context *ctx, FAT32_Info *fs, SD_FileName *file) {
  uint32_t current_cluster = fs->root_cluster;
  uint32_t cluster_to_free = 0;

  while (current_cluster >= 2 && current_cluster < 0x0FFFFFF8) {
    uint32_t lba = fs->data_start + (current_cluster - 2) * fs->sct_per_clus;

    for (uint8_t sct = 0; sct < fs->sct_per_clus; sct++) {
      if (sd_blk_read(ctx, lba + sct, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;

      for (uint16_t i = 0; i < 512; i += 32) {
        if (ctx->buffer[i] == 0x00) return SD_ERR_NOT_FOUND;
        if (ctx->buffer[i] == 0xE5) continue;

        if (memcmp(&ctx->buffer[i], file->name, 11) == 0) {
          // Pega o cluster (High word no offset 20, Low word no 26)
          cluster_to_free = ((uint32_t)*(uint16_t*)&ctx->buffer[i+20] << 16) |
                            (*(uint16_t*)&ctx->buffer[i+26]);

          ctx->buffer[i] = 0xE5; // Marca como deletado
          if (sd_blk_write(ctx, lba + sct, ctx->buffer) != SD_SUCCESS) return SD_ERR_WRITE;

          return sd_fat_free_chain(ctx, fs, cluster_to_free);
        }
      }
    }
```

```
    if (sd_fat_get_next_cluster(ctx, fs, current_cluster, &current_cluster) != SD_SUCCESS) break;
  }
  return SD_ERR_NOT_FOUND;
}

/**
 * @brief Converte uma string (ex: "teste.txt") para o formato 8.3 (ex: "TESTE   TXT").
 *
 * @param file_out Ponteiro para a estrutura SD_FileName que será preenchida.
 * @param name_in  String com o nome do arquivo (ex: "data.csv").
 * @return SD_Status SUCCESS ou ERR se o nome for inválido/longo.
 */
SD_Status sd_set_filename(SD_FileName *file_out, const char *name_in) {
  if (!file_out || !name_in) return SD_ERR_READY;

  // 1. Limpar a estrutura com espaços (padrão FAT)
  memset(file_out->name, ' ', 8);
  memset(file_out->ext, ' ', 3);

  uint8_t i = 0;
  // 2. Copiar o nome (até achar o ponto ou fim da string)
  while (name_in[i] != '.' && name_in[i] != '\0' && i < 8) {
    char c = name_in[i];
    if (c >= 'a' && c <= 'z') c -= 32; // Converter para maiúscula
    file_out->name[i] = c;
    i++;
  }

  // 3. Pular o ponto (se existir) e ir para a extensão
  const char *dot = strchr(name_in, '.');
  if (dot) {
    dot++; // Avança após o '.'
    for (uint8_t j = 0; j < 3 && dot[j] != '\0'; j++) {
      char c = dot[j];
      if (c >= 'a' && c <= 'z') c -= 32; // Converter para maiúscula
      file_out->ext[j] = c;
    }
  }

  return SD_SUCCESS;
}

/**
 * @brief Atualiza o setor FSInfo do FAT32.
 *
 * Escreve o número de clusters livres e o último cluster alocado para que
 * o sistema operacional não precise reescaneia o disco todo ao ser montado.
 *
 * @param ctx Ponteiro para o contexto SD.
 * @param fs Ponteiro para o sistema FAT32 montado.
 * @param free_clusters Qtd de clusters livres (use 0xFFFFFFFF se desconhecido).
 * @param last_alloc Último cluster alocado (ajuda na velocidade de busca).
```

```
 * @return SD_Status Status da gravação.
 */
SD_Status sd_fat_update_fsinfo(SD_Context *ctx, FAT32_Info *fs, uint32_t free_clusters, uint32_t
last_alloc)
{
   if (sd_blk_read(ctx, fs->fsinfo_lba, ctx->buffer) != SD_SUCCESS) {
      return SD_ERR_READY;
   }

   // Validação de Assinaturas (Offset 0, 484 e 510)
   // Essencial para evitar sobrescrever setores de dados por erro de LBA
   uint32_t lead_sig = *((uint32_t *)&ctx->buffer[0]);
   uint32_t struc_sig = *((uint32_t *)&ctx->buffer[484]);
   uint16_t trail_sig = *((uint16_t *)&ctx->buffer[510]);

   if (lead_sig != 0x41615252 || struc_sig != 0x61417272 || trail_sig != 0xAA55) {
      return SD_ERR_BOOT; // Estrutura FSInfo inválida ou setor errado
   }

   // Atualiza apenas se os valores forem lógicos
   if (free_clusters != 0xFFFFFFFF) {
      *((uint32_t *)&ctx->buffer[488]) = free_clusters;
   }

   *((uint32_t *)&ctx->buffer[492]) = last_alloc;

   if (sd_blk_write(ctx, fs->fsinfo_lba, ctx->buffer) != SD_SUCCESS) {
      return SD_ERR_WRITE;
   }
   return SD_SUCCESS;
}

/**
 * @brief Configura os registradores SPI do AVR para modo lento ou rápido.
 * @note Compatível com a maioria dos microcontroladores AVR (ATmega328P, ATmega2560, etc).
 * Dispositivos com mapeamento de registradores não padrão (ex: XMEGA) podem exigir
 * que o desenvolvedor forneça sua própria função set_speed na struct HAL.
 *
 * @param speed_mode SD_SPI_SLOW (fosc/64) ou SD_SPI_FAST (fosc/2)
 */
void sd_avr_spi_set_speed_default(uint8_t speed_mode) {
   if (speed_mode == SD_SPI_SLOW) {
      SPCR = (1<<SPE)|(1<<MSTR)|(1<<SPR1)|(1<<SPR0);
   } else {
      SPCR = (1<<SPE)|(1<<MSTR);
      SPSR |= (1<<SPI2X);
   }
}

/**
 * @brief Sincroniza um arquivo FAT32 no disco (flush)
 *
```

```
 * Atualiza o tamanho do arquivo na entrada de diretório,
 * garantindo consistência em caso de queda de energia.
 *
 * NÃO altera clusters nem força escrita de buffer externo.
 *
 * @param ctx Contexto SD
 * @param fs Sistema FAT32 montado
 * @param file Nome do arquivo (8.3)
 * @param current_size Tamanho atual do arquivo em bytes
 *
 * @return SD_Status
 */
SD_Status sd_file_sync(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file, uint32_t
current_size)
{
   uint16_t entry_offset;
   uint32_t entry_lba;

   // Localiza a entrada física do arquivo
   if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS) {
      return SD_ERR_NOT_FOUND;
   }

   // Atualiza o tamanho e força a escrita no cartão
   // O buffer já contém os dados do setor lido pela sd_find_file_entry
   *((uint32_t *)&ctx->buffer[entry_offset + 28]) = current_size;

   if (sd_blk_write(ctx, entry_lba, ctx->buffer) != SD_SUCCESS) {
      return SD_ERR_WRITE;
   }

   // Opcional: Atualiza o FSInfo para refletir clusters usados
   return sd_fat_update_fsinfo(ctx, fs, 0xFFFFFFFF, fs->next_free_cluster);
}

/**
 * @brief Lê o registrador CSD e calcula a capacidade total do cartão em Megabytes.
 *
 * @param ctx Contexto SD
 * @return uint32_t Capacidade em MB (ex: 7580 para um cartão de 8GB)
 */
uint32_t sd_get_capacity_mb(SD_Context *ctx) {
   uint8_t csd[16];
   uint32_t c_size;

   // CMD9: SEND_CSD
   if (sd_cmd_send(ctx, 9, 0, 0xFF) != 0x00) return 0;

   // Aguarda o token de dados 0xFE
   uint16_t timeout = 10000;
   while (ctx->hal.spi_tx_rx(0xFF) != 0xFE && --timeout);
   if (timeout == 0) return 0;
```

```
  // Lê os 16 bytes do CSD
  for (uint8_t i = 0; i < 16; i++) csd[i] = ctx->hal.spi_tx_rx(0xFF);

  // Descarta CRC
  ctx->hal.spi_tx_rx(0xFF); ctx->hal.spi_tx_rx(0xFF);
  ctx->hal.cs_high();

  // Cálculo para SDHC (Versão 2.0) - O mais comum hoje
  if ((csd[0] & 0xC0) == 0x40) {
    c_size = ((((uint32_t)csd[7] & 0x3F) << 16) | ((uint32_t)csd[8] << 8) | csd[9]) + 1;
    return c_size / 2; // (C_SIZE + 1) * 512KB / 1024 = MB
  }

  return 0; // Para simplificar, focamos em cartões modernos (SDHC/SDXC)
}

/**
 * @brief Lista todos os arquivos e pastas do diretório raiz.
 */
void sd_dir_list(SD_Context *ctx, FAT32_Info *fs) {
  uint32_t current_cluster = fs->root_cluster;
  uint8_t *entry;

  printf("\n--- Listagem do Diretorio Raiz ---\n");

  while (current_cluster >= 2 && current_cluster < 0x0FFFFFF8) {
    uint32_t lba = fs->data_start + (current_cluster - 2) * fs->sct_per_clus;

    for (uint8_t sct = 0; sct < fs->sct_per_clus; sct++) {
      if (sd_blk_read(ctx, lba + sct, ctx->buffer) != SD_SUCCESS) return;

      for (uint16_t i = 0; i < 512; i += 32) {
        entry = &ctx->buffer[i];

        if (entry[0] == 0x00) return; // Fim do diretório
        if (entry[0] == 0xE5) continue; // Arquivo deletado
        if (entry[11] & 0x0F) continue; // Pula LFN (Long File Name) ou Volume Label

        // Imprime o nome (8 caracteres)
        for (uint8_t j = 0; j < 8; j++) {
          if (entry[j] != ' ') putchar(entry[j]);
        }

        // Imprime a extensão se existir
        if (entry[8] != ' ') {
          putchar('.');
          for (uint8_t j = 8; j < 11; j++) putchar(entry[j]);
        }

        // Indica se é diretório
        if (entry[11] & 0x10) printf(" <DIR>");
```

```
            printf("\n");
        }
    }
    // Busca próximo cluster do diretório
    if (sd_fat_get_next_cluster(ctx, fs, current_cluster, &current_cluster) != SD_SUCCESS) break;
  }
}
```